

Министерство образования Республики Беларусь

Учреждение образования
БЕЛОРУССКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ИНФОРМАТИКИ И РАДИОЭЛЕКТРОНИКИ

Факультет информационных технологий и управления
Кафедра интеллектуальных информационных технологий

ОТЧЕТ
по лабораторной работе
по дисциплине «Естественно языковой интерфейс в интеллектуальных
системах»
на тему:

Автоматическое распознавание языка текста

Студенты гр. 321701

И. А. Политыко

В. А. Лукашов

Руководитель

Ю. Б. Крапивин

Минск 2026

СОДЕРЖАНИЕ

Перечень условных обозначений	3
1 Алгоритмы распознавания языка	4
2 Разработка системы распознавания языка	6
Заключение	12
Список использованных источников	13

ПЕРЕЧЕНЬ УСЛОВНЫХ ОБОЗНАЧЕНИЙ

ПОЯ	—	Поисковый образ языка
ПОД	—	Поисковый образ документа
ЕЯ	—	Естественный язык
ОоП	—	Расстояние Out-of-Place между ранжированными профилями
MLP	—	Многослойный перцептрон
N-грамма	—	Подстрока длины N
Ассигасу	—	Доля верно классифицированных текстов
F-мера	—	Гармоническое среднее точности и полноты
PDF	—	Переносимый формат документов
API	—	Программный интерфейс приложения
REST	—	Архитектурный стиль взаимодействия компонентов
JSON	—	Текстовый формат обмена данными
SQL	—	Язык структурированных запросов
СУБД	—	Система управления базами данных
ЛВС	—	Локальная вычислительная сеть
SPA	—	Одностраничное веб-приложение

1 АЛГОРИТМЫ РАСПОЗНАВАНИЯ ЯЗЫКА

По варианту 13 система определяет язык PDF-документа: русский или немецкий. Используются три метода построения и сравнения поисковых образов: коротких слов, частотных слов и нейросетевой (многослойный перцептрон на символьных N-граммах).

Общая схема совпадает с описанной в задании. Для каждого языка по обучающему корпусу строится поисковый образ языка (ПОЯ). Для входного документа по тем же правилам строится поисковый образ документа (ПОД). Стратегия сравнения выбирает язык с наибольшей оценкой принадлежности (для словесных методов — произведение вероятностей лексем, для нейросети — вероятность класса). Дополнительно между ранжированными профилями считается расстояние Out-of-Place.

Обучающий корпус лежит в каталоге `corpus/train`: по семь текстов медицинской тематики на каждый язык. Объём русской части составляет около 50 Кбайт, немецкой — около 85 Кбайт, что укладывается в требование задания (от 20 до 120 Кбайт). Эталонная разметка для оценки качества хранится отдельно в `qrels.json`: десять коротких фраз с заранее известным языком. Эти фразы не входят в обучение.

Текст сначала приводится к нижнему регистру, буква «ё» заменяется на «е». Токенизация допускает внутренние дефисы и немецкие умлауты. Русские словоформы лемматизируются анализатором `ru morphology3`; немецкие приводятся к нижнему регистру. Стоп-слова для построения ПОЯ не отбрасываются: служебные лексемы как раз различают языки.

Метод коротких слов включает в ПОЯ лексемы длиной не более пяти символов, встретившиеся в обучающем корпусе более трёх раз. Вероятность лексемы — отношение её частоты к сумме частот всех лексем профиля. Лексеме, отсутствующей в ПОЯ, назначается малая сглаживающая вероятность. Принадлежность документа языку оценивается как произведение вероятностей коротких лексем документа:

$$P(\text{lang} \mid \text{doc}) = \prod_{l \in L_{\text{short}}} P(l \mid \text{lang}). \quad (1.1)$$

На практике вычисляется сумма логарифмов, чтобы избежать потери точности. Выбирается язык с наибольшим значением. Профиль коротких слов шире топ-50, поэтому сравнение и расстояние ОоП на длинном документе занимают больше времени, чем у частотного метода.

Метод частотных слов строит ПОЯ из пятидесяти самых употребительных лексем корпуса без ограничения длины. С документом сравниваются только слова из объединения языковых пятёрок десятков. Оценка имеет тот же вид, что и (1.1), но по этому узкому словарю. Поэтому метод быстрее: после разбора текста почти вся работа сводится к просмотру

короткого списка.

Нейросетевой метод использует `MLPClassifier` из `scikit-learn`. Признак — бинарное присутствие символьной N -граммы ($N = 1, \dots, 5$) в тексте. В словарь признаков попадают N -граммы, встретившиеся не менее чем в двух обучающих документах. Сеть содержит два скрытых слоя (128 и 64 нейрона), функция активации — ReLU, оптимизатор — Adam. На выходе — две вероятности классов. Этот метод не строит словесный ПОЯ и не считает ОоП по лексемам.

Расстояние Out-of-Place сравнивает два списка, упорядоченных по убыванию частоты. Для каждой единицы профиля берётся модуль разности рангов; отсутствующему элементу назначается штраф, равный длине более длинного списка:

$$d(A, B) = \sum_{g \in A \cup B} |\text{pos}_A(g) - \text{pos}_B(g)|. \quad (1.2)$$

Абсолютное значение ОоП растёт вместе с длиной профилей, поэтому его нельзя напрямую сравнивать между методом коротких слов и методом топ-50. Решение о языке словесные методы принимают по вероятности (1.1), а не по минимуму ОоП.

Время каждого словесного метода измеряется от сырого текста: перед запуском сбрасывается кэш лемматизации, чтобы частотные слова не получали «бесплатную» предобработку после коротких слов. Нейросеть засекается отдельно по извлечению N -грамм и прогону сети.

2 РАЗРАБОТКА СИСТЕМЫ РАСПОЗНАВАНИЯ ЯЗЫКА

Система разворачивается в локальной вычислительной сети средствами Docker Compose и состоит из трёх сервисов: PostgreSQL 16, сервер FastAPI на Python 3.11 и одностраничный интерфейс на React за nginx.

Серверная часть использует FastAPI и Uvicorn, доступ к базе — `psycopg2`. Из PDF текст извлекается библиотекой `pypdf`. Русская лемматизация выполняется `pymorphy3`. Нейросеть — `MLPClassifier` из `scikit-learn`. Графики метрик строит `matplotlib` и сохраняет в каталог отчёта `png/`. Клиент — React 18, Vite и `react-router-dom`. Требование варианта о формате документов закрыто ограничением загрузки: на вход принимается только PDF.

Структура системы показана на рисунке 2.1. Модуль `main.py` объявляет REST-эндпоинты. `text_processing.py` выполняет токенизацию и лемматизацию. `language_model.py` реализует три метода и расстояние ОоП. `document_loader.py` читает PDF. `indexer.py` сохраняет документы и помечает источник: эталон из `qrels.json` или загруженный файл. `evaluator.py` считает Ассигасу, точность, полноту, F-меру и время. `bootstrap.py` при старте загружает обучающий корпус, обучает модели и индексирует эталонные фразы. `migrate.py` применяет SQL-миграции.

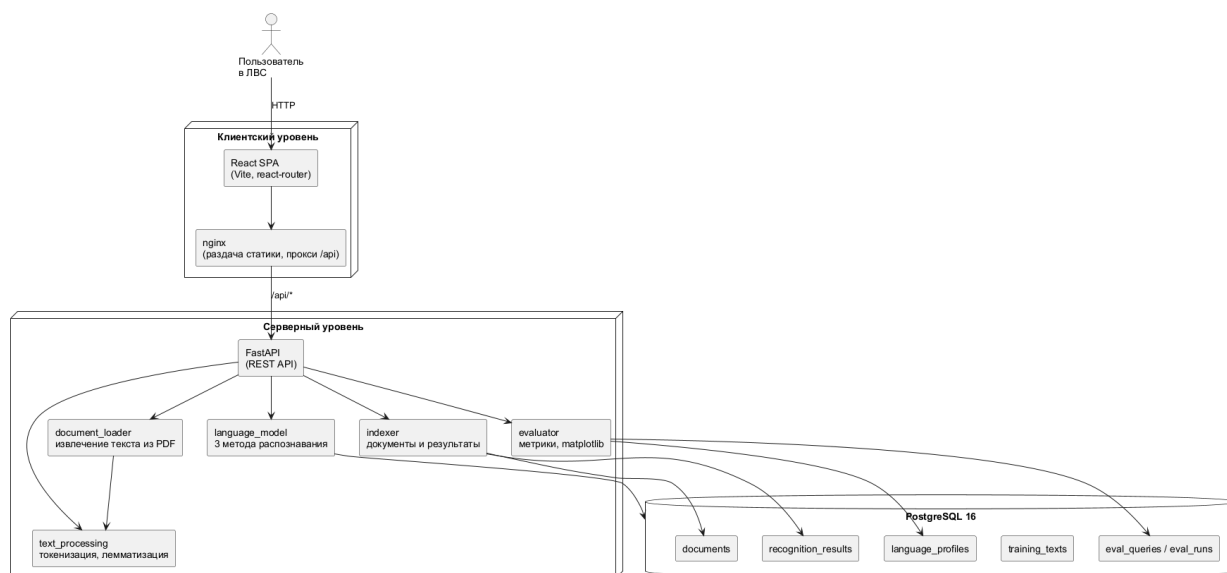


Рисунок 2.1 – Структура системы распознавания языка.

Последовательность инициализации при старте сервера показана на рисунке 2.2: корпус читается в `training_texts`, строятся ПОЯ и MLP, эталонные фразы из `qrels.json` попадают в `eval_queries` и в список документов как `eval_*.txt`.

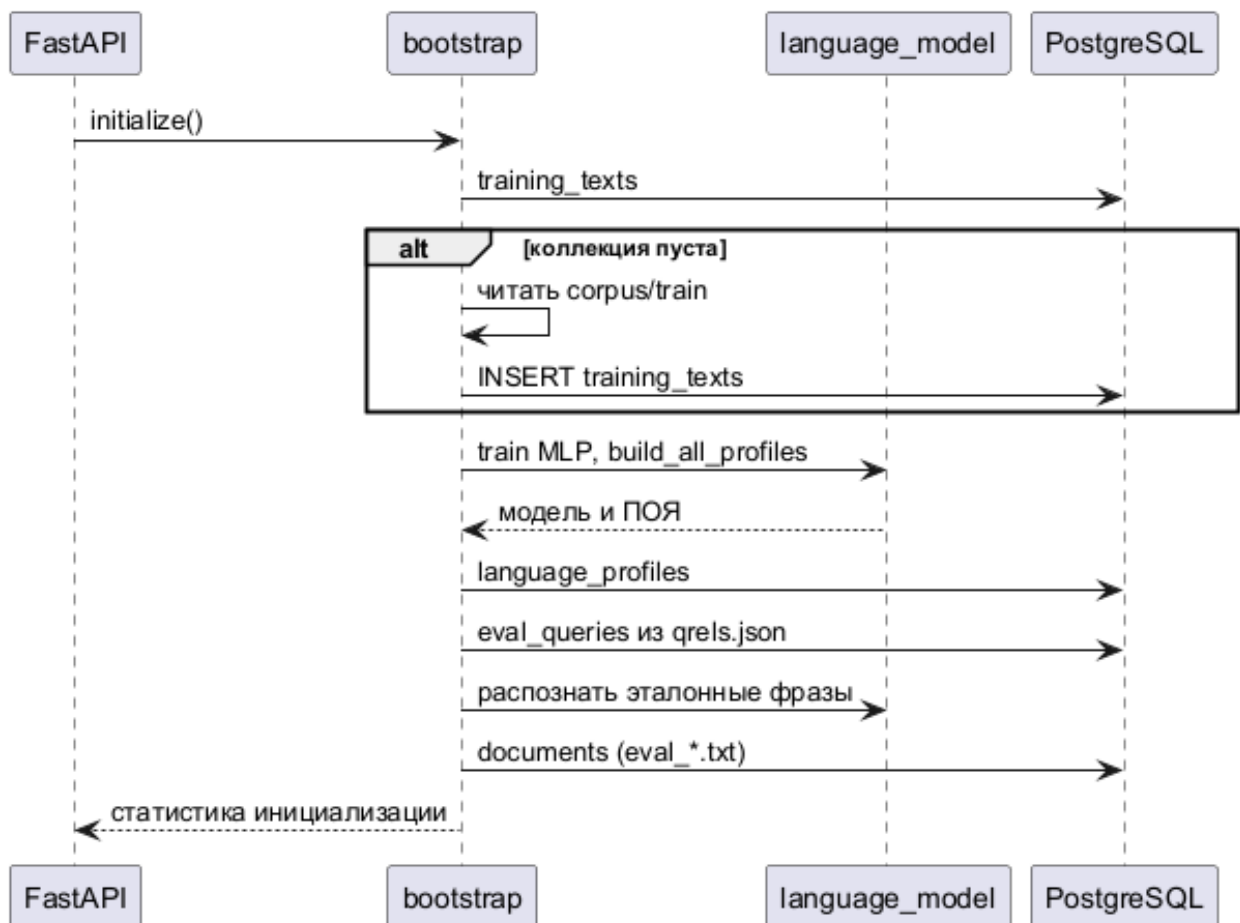


Рисунок 2.2 – Диаграмма последовательности обучения моделей и индексации эталонов.

Загрузка PDF и распознавание показаны на рисунке 2.3. Пользователь выбирает файл на странице распознавания; nginx передаёт запрос на POST /api/recognize; сервер извлекает текст, прогоняет три метода, записывает документ и возвращает язык, уверенность, ОоП и время.

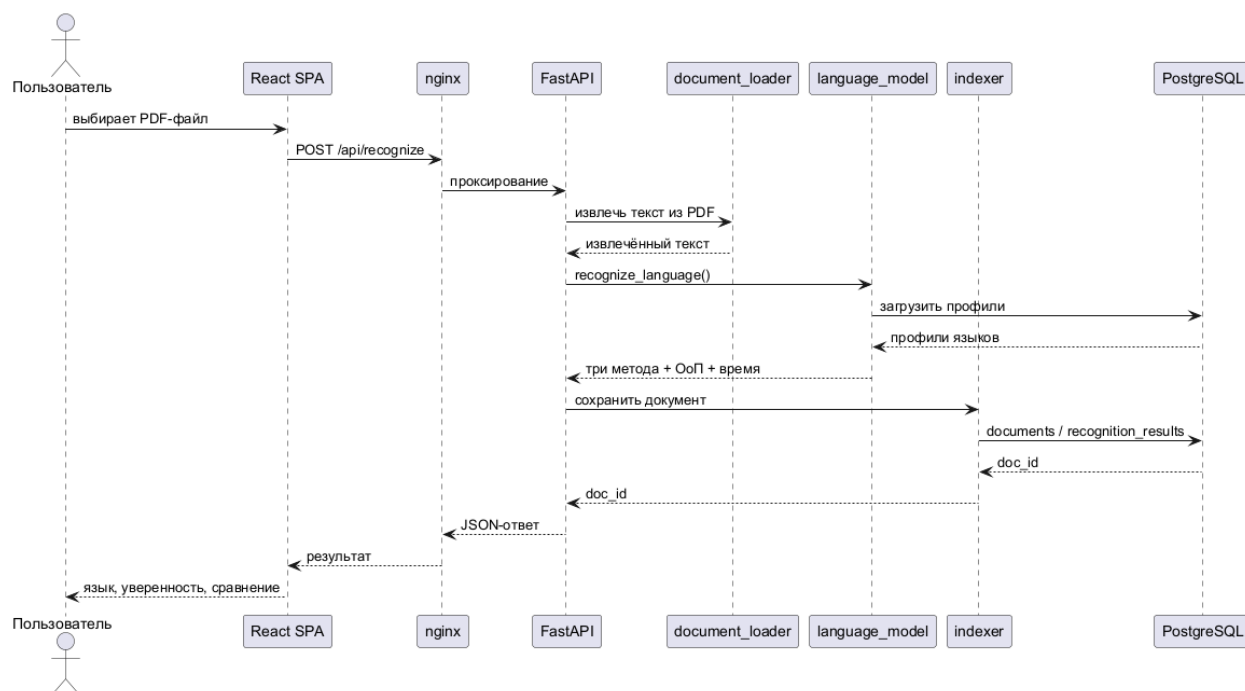


Рисунок 2.3 – Диаграмма последовательности распознавания PDF-документа.

Таблицы базы перечислены в таблице 2.1, схема — на рисунке 2.4. Обучающие тексты хранятся в `training_texts` и не попадают в пользовательский список. В `documents` лежат эталонные фразы (файлы `eval_*.txt`) и загруженные PDF.

Таблица	Назначение
<code>documents</code>	Заголовок, текст, язык, результат, источник
<code>training_texts</code>	Обучающие тексты по языкам
<code>language_profiles</code>	Лексемы и частоты ПОЯ по методам
<code>recognition_results</code>	Результат каждого метода по документу
<code>eval_queries</code>	Эталонные фразы и метка языка
<code>eval_runs</code>	Сводка прогона оценки

Таблица 2.1 – Таблицы базы данных системы распознавания языка.

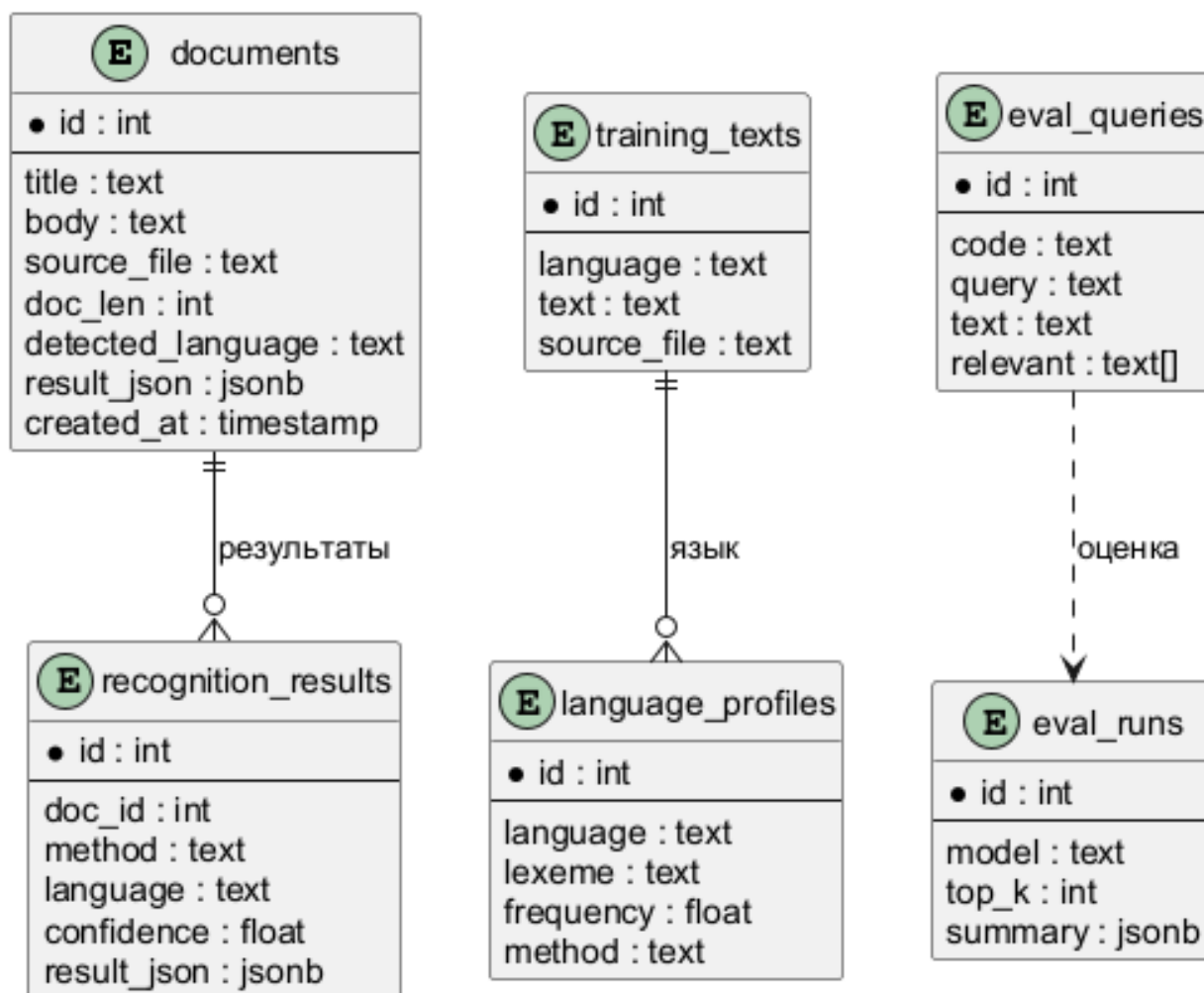


Рисунок 2.4 – Схема базы данных.

Интерфейс содержит страницы распознавания, списка документов с ссылкой на карточку, оценки качества и справки. Список документов можно сохранить в JSON и напечатать из браузера. Развёртывание: `docker compose up --build` из каталога программной части. Интерфейс доступен по адресу <http://localhost:8080>.

Проверка `scenarios.py` покрывает лемматизацию русского текста, сохранение немецких умлаутов, оба словесных метода, ОоП по рангам и обучение нейросети на короткой выборке. Все десять проверок прошли.

Оценка качества выполняется на десяти эталонных фразах (пять русских и пять немецких). Каждый метод классифицирует фразу, после чего считаются Ассигасу, макросредние точность, полнота и F-мера, а также среднее время. Результаты — в таблице 2.2. На этой выборке все три метода дали полную точность: русский и немецкий различаются алфавитом, и профили почти не пересекаются.

Метод	Асс	P	R	F1	мс
Коротких слов	1,000	1,000	1,000	1,000	0,26
Частотных слов	1,000	1,000	1,000	1,000	0,30
Нейросетевой	1,000	1,000	1,000	1,000	30,17

Таблица 2.2 – Метрики на эталонной выборке из десяти фраз.

На длинном PDF картина по времени другая: метод коротких слов обходит весь поток коротких лексем и широкий ПОЯ, частотный ограничен топ-50, нейросеть строит N-граммы по всему тексту. На демонстрационном документе получены величины порядка 355 мс, 58 мс и 403 мс соответственно. Сравнение методов по качеству и по времени показано на рисунках 2.5 и 2.6.

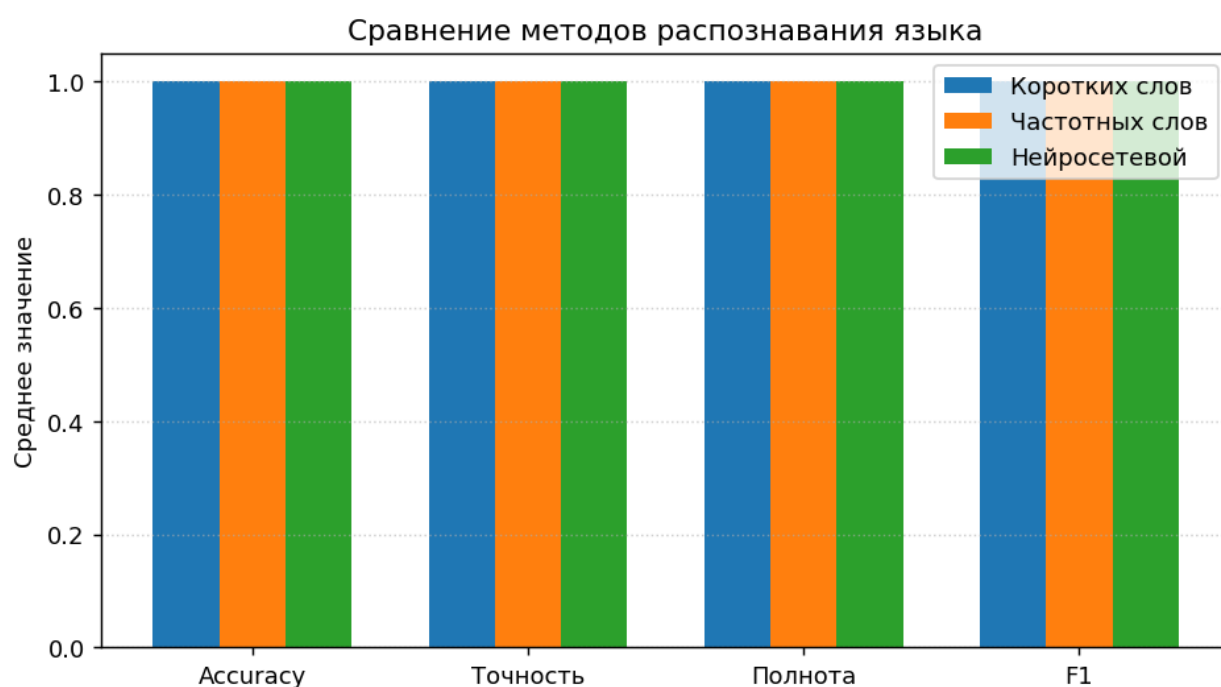


Рисунок 2.5 – Сравнение методов по Ассигасу, точности, полноте и F-мере.

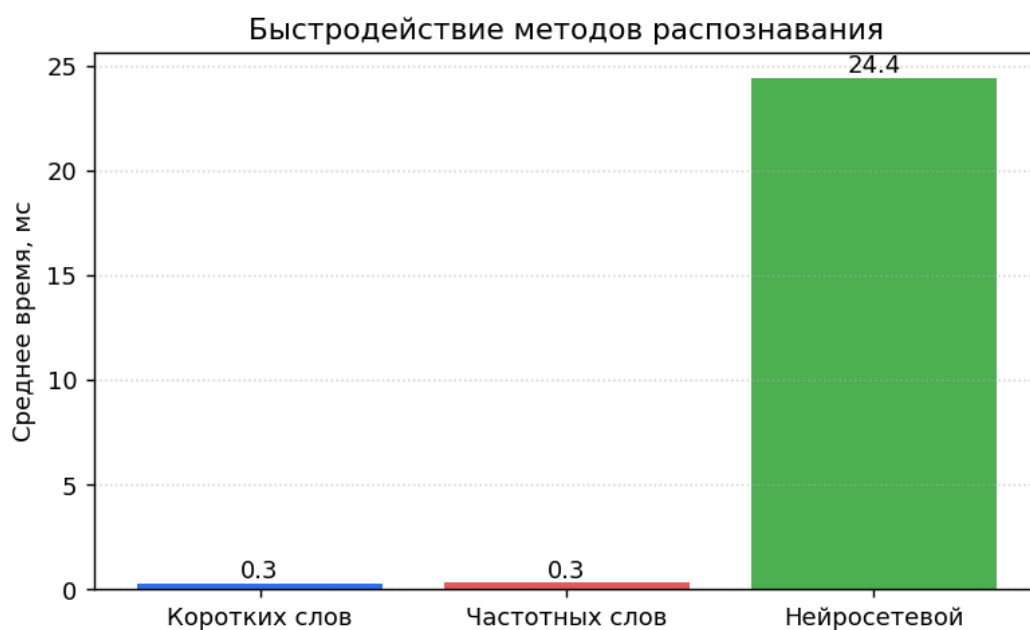


Рисунок 2.6 – Среднее время методов на эталонных фразах.

Для ручной проверки подготовлены четыре одностраничных PDF в каталоге `demo-pdf`: два русских (`ru_priem.pdf`, `ru_operaciya.pdf`) и два немецких (`de_aufnahme.pdf`, `de_operation.pdf`). Их можно загрузить на странице распознавания.

ЗАКЛЮЧЕНИЕ

Реализована система автоматического распознавания языка PDF-документа для русского и немецкого языков. По варианту 13 использованы метод коротких слов, метод частотных слов и нейросетевой метод на бинарных символьных N-граммах. Система разворачивается в ЛВС средствами Docker Compose.

Обучающий корпус составлен из четырнадцати медицинских текстов. Эталонная выборка из десяти фраз дала Ассигасу 1,000 у всех трёх методов. На длинном документе быстрее оказывается метод частотных слов: его ПОЯ ограничен пятьюдесятью лексемами. Метод коротких слов и нейросеть обрабатывают более широкое пространство признаков и поэтому занимают больше времени.

Перспективы: добавить языки с общей латиницей, где словесные профили пересекаются сильнее; подобрать глубину N-грамм под объём корпуса; вынести обучение нейросети в отдельный этап, чтобы не повторять его при каждом старте контейнера.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

- [1] Cavnar W., Trenkle J. N-Gram-Based Text Categorization // Proceedings of SDAIR-94. — 1994. — Режим доступа: <https://www.let.rug.nl/vannoord/TextCat/textcat.pdf>. — Дата доступа: 23.09.2026.
- [2] Pedregosa F. и др. Scikit-learn: Machine Learning in Python // Journal of Machine Learning Research. — 2011. — Vol. 12. — P. 2825–2830.
- [3] FastAPI documentation [Электронный ресурс]. — Режим доступа: <https://fastapi.tiangolo.com/>. — Дата доступа: 23.09.2026.
- [4] Korobov M. Morphological Analyzer and Generator for Russian and Ukrainian Languages // Analysis of Images, Social Networks and Texts. — 2015. — P. 320–332.
- [5] rypdf [Электронный ресурс]. — Режим доступа: <https://rypdf.readthedocs.io/>. — Дата доступа: 23.09.2026.
- [6] PostgreSQL 16 Documentation [Электронный ресурс]. — Режим доступа: <https://www.postgresql.org/docs/16/>. — Дата доступа: 23.09.2026.
- [7] React documentation [Электронный ресурс]. — Режим доступа: <https://react.dev/>. — Дата доступа: 23.09.2026.
- [8] MLPClassifier [Электронный ресурс]. — Режим доступа: https://scikit-learn.org/stable/modules/generated/sklearn.neural_network.MLPClassifier.html. — Дата доступа: 23.09.2026.